HW3: Simple MPI Circle Renderer

Circles — example rendered outputs

1. Introduction

This assignment implements a simple MPI-based circle renderer. The program reads a binary scene description, rasterizes circles to an image, and writes a PNG output. The goal is to parallelize the renderer (MPI), measure performance, and analyze scaling and load balance.

Data model (per circle):

- `float3 position` — 2D position (x, y) on the image plane
- `float radius` — circle radius
- `float3 color` — RGB color

Algorithm (high level):

```
clear image buffer
for each circle (back-to-front order):
    compute screen-space bounding box
    for each pixel in bounding box:
        compute pixel center
        if pixel center inside circle:
            compute circle color contribution
            blend into pixel
```

The provided skeleton includes readers and writers for the binary format; you should not need to modify file I/O unless stated.

Note: the binary scene file stores circles in drawing order — entries are listed from back to front (file order = back -> front). If you read the file in sequence, you are processing circles back-to-front.

Binary file format

The renderer expects a compact binary scene format (little-endian). Fields are:

- Header:
  - 4 bytes: magic ASCII `CRDR`
  - 4 bytes: version (uint32)

    - 8 bytes: count (uint64) — number of circle records
    - 6 floats: bbox as float32 (xmin, ymin, zmin, xmax, ymax, zmax)

- Per-record layout (packed, little-endian):
    - `float32 x` — circle center X
    - `float32 y` — circle center Y
    - `float32 radius` — circle radius
    - `uint8 r, g, b` — color channels (0..255)

Each record is therefore 3*4 + 3 = 15 bytes (no depth `z` per-record and no alpha channel). Files list circles in back-to-front drawing order (earlier records are farther back).

Notes:

- The header's bbox can be used to infer image size when the bbox represents pixel-space (e.g., xmin=0, ymin=0, xmax=W, ymax=H).
- All numeric fields are little-endian (matching the provided generators and readers).

2. Requirements

1. Implement a correct serial renderer (single-process).
2. Parallelize the renderer with MPI
3. Report and analyze the results.

General Tips About Your Report

- Describe the input sizes and how they affect runtime (image size, number of circles).
- Report runtime vs. number of processes (strong scaling).
- Report runtime vs. problem size per process (weak scaling).
- Report speedup and efficiency relative to the serial baseline.
- What optimization techniques you applied?
- Anything you want to share.

Note

- Look up the per-rank timing and load statistics in your experiment logs. Did you observe load imbalance? If so, quantify it (e.g., max/mean time, standard deviation) and it might help you out stand in the competition.

3. Environment

- **OS**: Ubuntu 24.04 Server
- **CPUs**: 144
- **Sockets**: 2
- **Cores per socket**: 36
- **Threads per core**: 2
- **Networking**: All nodes connected via Mellanox ConnectX-5 Adapter to the same switch
- **Total Nodes**: 8

Node Allocation and Restrictions

To accommodate all students, the class is divided into two groups:

- **Group 1 (Team 1 -- Team 25)**: Node 0 -- Node 3
- **Group 2 (Team 25 -- Team 50)**: Node 4 -- Node 7

Restriction: Each team should not create more than **16 computing processes/threads per node** to minimize interference.

Connection Information (SSH)

- **MPI Nodes IP**: `172.16.179.50` -- `172.16.179.57`
- **Jump Server**: `140.112.90.37` (Port: 9037)
  - *NTU CSIE Students*: Can directly log in to MPI Nodes if your client is in the `172.16.0.0/16` subnet. Connect to the CSIE VPN first.
  - *Other Students*: Log in to the Jump Server first, then access MPI Nodes from there.

Warning: Do NOT execute computing processes on the Jump Server!

4. Execution

MPI Setup

> You should finish this part to run your MPI program.

1. Generate an SSH key pair on one of the MPI node (leave the passphrase empty):

   ```
   ssh-keygen -t ed25519
   ```

2. Make all the nodes you are allowed to access recognize your public key:

   ```
   ssh-copy-id -i ~/.ssh/id_ed25519.pub rdmaX
   ```

   `rdmaX` is the host name of `172.16.179.4X` recognizable among the MPI nodes. Remember to change the `X` into some number.
3. Copy the private key to all the nodes you are allowed to access:

   ```
   scp ~/.ssh/id_ed25519 rdmaX:.ssh
   ```

> It is recommended to just leave your private key on the MPI nodes for security's sake. For accessing from outside, generate another key pair on your PC or the jump server and run `ssh-copy-id` to send your public key to the MPI nodes.

MPI Run

> You should place your code and testing data at the same absolute path on all MPI nodes. Remember to copy the testing data on all MPI nodes beforehand. Besides, after compiling your code with `mpicc`, remember to copy your compiled code to all MPI nodes under the same path.
>
> By the way, though the testing data are placed on all nodes, only the rank 0 process will read the data, and you are asked to distribute them from the rank 0 process.

We will run your code like this:

```
# Compile (linking math library and enabling CPU-specific optimizations)
mpicc renderer_mpi.c -o renderer_mpi -lm -march=native

# Copy binary to other nodes (example for nodes rdma1..rdma3)
for i in 1 2 3; do scp renderer_mpi rdma$i:~/; done

# Run on 4 nodes with 16 processes per node
UCX_TLS=rc,sm,self \
UCX_NET_DEVICES=mlx5_0:1 \
UCX_LOG_LEVEL=info \
mpirun \
  --hostfile hosts \
  -npernode 16 \
  --mca pml ucx \
  --mca btl ^tcp \
  ./renderer_mpi <testing_data>
```

Content of `hosts`:

```
rdma0
rdma1
rdma2
rdma3
```

For testing, you can try different configurations of the host file. Just remind you here that different teams are allowed to access different group of nodes.

5. Submission

- **Deadline**: 4/24 (Fri) 23:59
- **Submission**: One team member uploads `Team_<Team Number>_HW3.zip`
- **Required files**:
  1. `renderer.c` — serial implementation
  2. `renderer_omp.c` — final optimized implementation
  3. `Team_<Team Number>_HW3_report.pdf`

Please make sure your source codes can be compiled and executed without any problem. If some modifications are required to compile and run your code, there will be a small points deduction.

6. Grading Policy

1. **Correctness (10%)**
    - The points you earned in this part is proportional to the number of test cases you passed.
    - No differences from golden output (diff == 0) for correctness.
2. **Scalability (10%)**
    - You will get full points as long as your parallelized code can show good scalability.
    - Please justify the scalability of your program in the report.
3. **Competition Rank (40%)**
    - Score formula: `score = max(10, 40 × (100 − 5 × (rank − 1))%)`.
    - However, you will get 0% in this part if your program gives wrong answers.
4. **Report (40%)**
    - The more thorough, reasonable, and clearer your analysis is, the more points you will get in this part.

⚖️ **Fairness & Compilation Standard**: To ensure absolute fairness across all submissions, **grading will strictly use the compilation and execution commands provided in Section 4**. No additional flags, custom compilers, or environment-specific optimizations outside the provided commands will be applied during the official ranking and scoring process.

Warning:

- Submissions after the deadline will have their total score multiplied by **70%**
- Follow the template code instructions and do not modify code that should not be changed